Laboratorio 2, comentado

Unidad 2 — Sesión 1 — Las columnas que dejamos afuera, y lo que abren

Dr. Carlos Hernández Ulloa
Reiner Romero Garrido

Facultad de Ingeniería — Universidad San Sebastián

Qué es este documento

Junto con estas láminas se publica **el cuaderno resuelto y ejecutado**:

`Semana01_Lab2_Columnas_Que_Dejamos_Afuera_SOLUCION_Ejecutado_en_COLAB.ipynb`

El cuaderno muestra **qué salió**. Estas láminas dicen **qué significa**, y agregan las **respuestas escritas** de Q3, Q4 y Q6, que quedaron en blanco a propósito.

> *Y hay una tercera cosa, que ocupa el último tercio: este laboratorio dejó una pregunta sin responder.* ***El martes 25 la clase entera arranca de ahí.***

Todas las cifras están verificadas dos veces

Dónde se corrió	Para qué
Google Colab	es el entorno en que ustedes trabajaron: mismas versiones, mismo resultado
El computador del curso	para que ninguna cifra de este PDF dependa de una sesión de Colab ya cerrada

Las dos corridas dan **exactamente los mismos números**, hasta el tercer decimal. Es lo que garantiza la semilla fija de `generar_datos_coronel()`.

> *Si a alguno le salió distinto, no es el azar: es una celda ejecutada fuera de orden.*

De dónde viene este cuaderno

Una sola pregunta, en tres tiempos

	Pregunta	Respuesta
Lab 1	¿cuánto explican las cuatro columnas numéricas?	R^2 **0.579** — falta el 42%
Lab 2	¿dónde estaba ese 42%?	en las cinco columnas de texto: R^2 **0.884**
Sesión 4	¿y cuando el texto trae 50.000 niveles?	*el martes*

> *Los dos laboratorios no son «una práctica de Scikit-learn» y después «otra de codificación». Son **el mismo problema, dos veces**, y la segunda vez deja la puerta abierta.*

El mapa de los siete bloques

	Pregunta	Lo que se mide
Q1	¿cuánto valía lo que botamos?	R^2 0.579 → 0.874
Q2	¿por qué limpiar *antes* de codificar?	45 columnas → 37
Q3	¿una categoría es un número?	**entero 0.596 · one-hot 0.804**
Q4	¿y cuando sí hay orden?	0.582 · 0.628 · 0.628
Q5	¿cuánto cuesta el one-hot?	9 columnas → 42 → 37
Q6	¿para qué sirve regularizar?	depende del tamaño, no del prestigio
Q7	¿y en producción?	`ValueError`, y cómo se evita

> *Los siete están encadenados: cada uno es consecuencia del anterior. El one-hot de Q5 es el que **crea** el problema que Q6 resuelve.*

Q1 — Lo que valían las cinco columnas de texto

	Columnas	R^2	MAE
Solo numéricas *(Lab 1)*	4	0.579	1.706 h
Con las categóricas	37	**0.874**	**0.944 h**

El error medio bajó de 1 hora 42 minutos a 57 minutos. Y el modelo es exactamente el mismo `LinearRegression`, con las mismas 594 filas.

> *El salto más grande de todo el semestre no vino de cambiar de modelo, sino de **darle al modelo información que ya estaba en la tabla**. Esa es la razón por la que preparar datos ocupa más tiempo que modelar en casi cualquier proyecto real.*

Q2 — Limpiar antes de codificar

```
niveles de tipo_carga: ['Granel', 'Peligroso', ..., 'granel', 'peligroso', ...]
niveles de prioridad : [' alta', ' baja', ..., 'alta', 'baja', ...]
```

Ocho niveles donde hay cuatro. Para `get_dummies`, `"Seco"` y `"seco"` son dos columnas distintas, cada una con la mitad de las filas.

	Columnas tras one-hot	R^2
Sin limpiar	45	0.874
Limpio	**37**	**0.884**

> ***El modelo no falló ni avisó.*** *Entrenó, predijo y entregó un número razonable. Un faltante se ve con `.isna()` y un atípico con `.describe()`; una categoría partida en dos* ***solo se ve mirando la lista de niveles****, que es lo que nadie hace.*

Q3 — El núcleo: una categoría no es un número

`tipo_carga`	Código que asigna `sorted()`	Horas que suma de verdad
granel	0	+1.2
peligroso	1	**+5.0**
refrigerado	2	+3.5
seco	3	**+0.0**

Entero: R^2 0.596. One-hot: R^2 0.804.

Con un entero, el modelo lineal solo puede aprender **un** coeficiente para esa columna, y con eso afirma que el efecto avanza parejo por la lista: que refrigerado suma el doble que peligroso y seco el triple. Traza la recta menos mala entre cuatro puntos que no están en línea.

> *El one-hot le da **cuatro efectos independientes** y ninguna suposición de orden.*

Respuesta Q3 — ¿por qué una codificación equivocada igual ayuda?

Porque no toda la información se destruye: queda la que sobrevive al orden inventado.

`sorted()` puso granel=0, peligroso=1, refrigerado=2, seco=3. Los efectos reales son +1.2, +5.0, +3.5, +0.0. Ese orden **no es el correcto**, pero tampoco es aleatorio respecto del efecto: los dos códigos del medio son los dos tipos caros, y el último es el barato.

Modelo	R^2
Sin la columna	0.579
Columna como entero	0.596
Columna con one-hot	**0.804**

> *El entero rescata 0.017 de los 0.225 disponibles: **el 7%**. Que una mala decisión mejore algo el número es exactamente lo que la vuelve difícil de detectar — si hubiera empeorado, se habría notado enseguida.*

Q4 — Cuando el orden sí existe

Codificación de `prioridad`	Columnas	R^2
Entero, orden alfabético (`alta=0, baja=1, media=2, urgente=3`)	1	0.582
Entero, orden correcto (`baja=0, media=1, alta=2, urgente=3`)	1	**0.628**
One-hot	3	**0.628**

Dos lecturas, y la segunda es la que importa:

1. `sorted()` **no sabe nada del negocio.** Pone «alta» antes que «baja» porque la *a* va antes que la *b*. El orden correcto hay que escribirlo a mano, y para escribirlo hay que conocer el dominio.
2. **El ordinal correcto y el one-hot empatan exactamente.** El ordinal no ganó precisión: logró lo mismo **con una columna en vez de tres**.

> *El criterio no es «categórica o numérica», sino:* ***¿hay un orden real, y el efecto avanza parejo a lo largo de ese orden?***

Respuesta Q4 — ¿`altura_pila` es numérica, ordinal o nominal?

Va de 1 a 5, así que nominal no es. La pregunta real es si el efecto avanza parejo, y eso se comprueba, no se opina:

```
# el efecto medido nivel por nivel, contra el que supone la recta
X_oh = pd.get_dummies(datos[NUMERICAS + ["altura_pila"]], columns=["altura_pila"])
```

Cómo la trató el Lab 1	Qué supuso
Como numérica, un coeficiente	que de 1 a 2 cuesta lo mismo que de 4 a 5

Cómo comprobarlo: codificarla con one-hot y comparar el R^2. Si el one-hot gana claramente, el efecto no es parejo y la recta estaba forzando. Si empatan —como pasó con `prioridad`— la columna numérica basta y es más barata.

> *En estos datos el efecto es **exactamente** lineal, porque se generó con `0.80 * altura_pila`. Con datos reales, esa comprobación es obligatoria.*

Q5 — El precio del one-hot

```
partimos de 9 columnas
drop_first=False -> 42 columnas
drop_first=True  -> 37 columnas
```

Columna	Niveles	Columnas que aporta
`naviera`	15	**14**
`terminal`	12	**11**
`tipo_carga` · `prioridad`	4 y 4	3 y 3
`turno`	3	2

Dos columnas se llevan 25 de las 33. El precio del one-hot no lo fija la cantidad de columnas de texto: lo fija la **cardinalidad**, es decir, cuántos valores distintos tiene cada una.

> *Guarden ese número: **37 columnas con 594 filas.** Vuelve en Q6, y vuelve el martes.*

Q5 — Y `drop_first`, que no hace lo que dicen

Las cuatro columnas de `tipo_carga` **suman exactamente 1 en toda fila**: conocidas tres, la cuarta se deduce. Esa suma constante es redundante con el intercepto.

	R^2	Coeficiente de mayor magnitud
`drop_first=False` (42 columnas)	**0.884**	2.49
`drop_first=True` (37 columnas)	**0.884**	3.68

El R^2 es idéntico. Hay que decirlo sin adornos, porque contradice cómo suele enseñarse: `drop_first` **no mejora la predicción**. Mínimos cuadrados resuelve igual con las columnas redundantes.

> *Lo que sí cambia son los coeficientes. Con columnas redundantes hay **infinitas** combinaciones que dan la misma predicción y el ajuste entrega una cualquiera. `drop_first` importa cuando el objetivo es **interpretar**, no cuando es predecir.*

Q6 — Regularizar: la decepción, primero

Modelo, con las 445 filas de entrenamiento	R^2
OLS, sin regularizar	**0.884**
Ridge α=1	0.884
Ridge α=10	0.884
Ridge α=50	0.876
Ridge α=200	**0.803**

No aporta nada, y con α grande empeora. Con 445 filas y 37 columnas hay **doce filas por columna**: datos de sobra. El modelo sin regularizar no se está sobreajustando, así que no hay nada que corregir.

> *Castigar coeficientes grandes cuando no hay sobreajuste es sesgar la estimación a cambio de nada.*

Q6 — Y cuándo sí sirve

Filas de entrenamiento	OLS	Ridge α=10	Lasso α=0.05
40	**−0.147**	0.570	**0.718**
60	0.683	0.740	0.797
100	0.752	0.779	0.829
200	0.862	0.869	0.881
445	0.884	0.884	0.884

Con 40 filas y 37 columnas, OLS da R^2 negativo: predice peor que responder siempre el promedio. Lasso, con exactamente los mismos datos, llega a 0.718.

> *La regularización no es una buena práctica que se aplica siempre: es **un seguro contra tener pocas filas en relación a las columnas**. Y esas 37 columnas salieron del one-hot de Q5 — es el propio one-hot el que fabrica la condición que la hace necesaria.*

Q6 — Lasso encuentra la columna inútil, solo

```
columnas de naviera: 14 | anuladas por Lasso: 14
```

Coeficiente	Magnitud
`dias_en_patio`	1.992
`altura_pila`	1.071
`tipo_carga_peligroso`	1.064
`tipo_carga_refrigerado`	0.763

Las 14 columnas de `naviera` quedaron en cero exacto, y ninguna de `tipo_carga` se tocó. Y tiene razón: al generar los datos, `naviera` se sorteó al azar, sin ningún efecto sobre las horas.

> *Lasso lo descubrió **a partir de los datos**, sin que nadie se lo dijera. Esa es su virtud práctica, y es distinta de mejorar el R^2.*

Respuesta Q6 — la `naviera` de tu proyecto

Se reconoce por dos síntomas juntos: muchos niveles y ningún mecanismo que explique por qué debería predecir.

Sospechosa típica	Por qué
Número de folio, orden de compra, guía de despacho	identifica, no describe
RUT de cliente en un maestro de miles	cada nivel tiene tres o cuatro filas
Código de operador o de usuario que digitó	mide quién escribió, no qué pasó

> ***El caso peligroso es el contrario**: la columna que sí predice y no debería. Si el código de operador predice bien, puede ser que un turno haga las cosas distinto — o que ese operador registre los casos después de saber el resultado. Lo segundo se llama fuga, y hunde el modelo el día que entra en producción.*

Q7 — Lo que se rompe en producción

```
nuevo["naviera"] = "NAV-99"        # una naviera que nunca apareció
pd.get_dummies(nuevo, columns=CATEGORICAS, drop_first=True)
```

```
columnas del modelo: 37 | columnas del caso nuevo: 4
❌ The feature names should match those that were passed during fit.
```

`get_dummies` **decide las columnas mirando los datos que le pasan.** Sobre una fila produce 4 columnas; sobre 594 produce 37. Nunca dos veces lo mismo.

> *No es un detalle de ingeniería: es que la transformación **no es parte del modelo**, y el día que llega un caso nuevo, nadie recuerda cuáles eran las 37 columnas ni en qué orden iban.*

Q7 — La tubería, y la fuga que no fue

```
ColumnTransformer([
    ("num", Pipeline([SimpleImputer(strategy="median"), StandardScaler()]), NUMERICAS),
    ("ord", OrdinalEncoder(categories=[["baja","media","alta","urgente"]]), ["prioridad"]),
    ("nom", OneHotEncoder(handle_unknown="ignore", drop="first"), NOMINALES),
])
```

R^2 **0.880**, y con `NAV-99` predice **11.42 horas** en vez de reventar. Es Q3 y Q4 convertidos en configuración: cada tipo de columna con su tratamiento, aprendido en el `fit` y aplicado igual siempre.

Y el experimento de la fuga dio 0.00000. Las dos medianas valen 10.0, así que imputar con todos los datos fue inofensivo **en este conjunto**.

> *La razón para usar la tubería no es ese experimento —que no mostró nada— sino el `ValueError` de la lámina anterior. La fuga sí importa en series temporales, en columnas con muchos faltantes y en conjuntos chicos.*

Laboratorio 2 — el saldo

Decisión	Efecto medido
Incluir las columnas categóricas	R^2 **0.579** → **0.874**
Regularizar, con 40 filas	R^2 **−0.147** → **0.718**
One-hot en vez de entero, en una nominal	R^2 **0.596** → **0.804**
Orden ordinal correcto en vez de alfabético	R^2 0.582 → 0.628
Limpiar mayúsculas y espacios	R^2 0.874 → 0.884, y 8 columnas menos
`drop_first`	**ninguno** al predecir; sí al interpretar
Regularizar, con 445 filas	**ninguno**
Evitar la fuga al imputar	**0.00000**

> *Tres de las ocho no cambiaron nada medible.* ***Y cuáles caen en cada grupo depende del conjunto de datos, no del prestigio de la práctica.***

Lo que queda abierto

De aquí arranca el martes 25

La pregunta que Q5 contestó a medias

> ***¿Cuánto cuesta el one-hot?***

La contestamos con **cinco columnas de texto y 15 niveles como máximo**: nueve columnas se volvieron 37, el modelo aguantó sin despeinarse y Lasso limpió lo que sobraba.

A esa escala, la respuesta es **«casi nada»**. Y es una respuesta correcta que se vuelve falsa al cambiar un solo número.

> *Un maestro de materiales de Inchalam tiene **50.000 códigos**. La lista de clientes de Pacific-Blue, decenas de miles. Y las palabras de un idioma, más todavía.*

Primero, de dónde salieron esas 37 columnas

	Filas	Columnas	Filas por columna
Lab 1 — solo numéricas	445	4	**111**
Lab 2 — con las categóricas	445	37	**12**
Q6 — el caso que se cayó	40	37	**1.1**

Las columnas no las puso el modelo: las puso la cardinalidad de los datos. `naviera` y `terminal`, con 15 y 12 niveles, aportaron 25 de las 33 columnas nuevas.

> *El one-hot es la única receta que el laboratorio enseñó, y su costo **crece uno a uno con los niveles**. La pregunta siguiente es qué le pasa a esa tabla de tres filas cuando la cardinalidad sube.*

Medido: una columna sin ninguna información

Un identificador administrativo agregado a las 37 columnas, con **cero efecto** sobre las horas — el caso de `naviera`, a escala creciente. Promedio de diez repeticiones:

Niveles	Columnas	OLS	Lasso	Filas de prueba con un nivel nunca visto
15	51	0.878	0.883	0 de 149
150	184	0.817	0.871	8
450	371	0.730	0.867	57
5.000	596	0.705	0.876	**137**

La columna no dice absolutamente nada, y aun así hunde el modelo 17 puntos. Lo único que cambió fue cuántas columnas cuesta escribirla.

> *La caída no es pareja —lo que se está midiendo es ruido de estimación, y salta— pero la dirección es inequívoca. Y Lasso aguanta plano:* ***regularizar es lo que sobrevive a la cardinalidad.***

Medido: la misma columna, ahora con señal

Mismo experimento, pero el identificador **sí tiene efecto propio por nivel**. Tres formas de tratarlo:

Niveles	Filas por nivel	Ignorarlo	One-hot	Una columna densa
15	29.7	0.786	0.891	0.896
50	8.9	0.775	0.880	0.897
150	3.0	0.758	0.824	0.892
300	1.5	0.758	0.764	0.897
594	**0.75**	**0.773**	**0.696**	**0.896**

A 300 niveles el one-hot ya no le gana a tirar la columna a la basura. A 594 es peor que ignorarla — y la información sigue estando ahí entera, como muestra la última columna.

> *La última columna es la trampa: le entregamos al modelo el efecto verdadero, ya resumido en un número por fila. No es un modelo entrenable; es la cota de lo que se podría lograr.*

Por qué el one-hot se cae, exactamente

No se queda sin memoria. Se queda sin filas por nivel.

Con 594 niveles y 445 filas de entrenamiento, cada nivel aparece en **menos de una fila** — y hay que estimarle un coeficiente propio.

Y hay un segundo golpe, que ya vieron en Q7: el `ValueError` de `NAV-99` no era un problema de programación. Con 5.000 niveles, **137 de las 149 filas de prueba traen un nivel que el entrenamiento nunca vio**. `handle_unknown="ignore"` evita la excepción; no evita que el modelo no tenga nada que decir sobre esas filas.

> *Las dos fallas tienen la misma causa: **una columna por nivel obliga a estimar un parámetro por nivel**, y los niveles crecen más rápido que los datos.*

Lo que falta tiene nombre, y es la clase del martes

	One-hot	Lo que hace falta
Columnas de entrada	una por nivel	**unas pocas, siempre las mismas**
Parámetros por nivel	uno, estimado con sus propias filas	uno por dimensión, **compartido**
Niveles parecidos	no existen: todos equidistan	**cerca en el espacio**

Con 50.000 niveles y una capa de 128:

	Parámetros
One-hot + capa densa	50.000 × 128 = **6.400.000**
Tabla densa de dimensión 32	50.000 × 32 + 32 × 128 ≈ **1.600.000**

> *Esa tabla densa y entrenable se llama **embedding**, y es la primera capa de todo modelo de lenguaje. El martes empezamos ahí.*

Qué del martes se apoya en este laboratorio

De estas láminas	A la Sesión 4
`naviera`: 15 niveles, 14 columnas de ruido	el maestro de materiales: **50.000 códigos**
Q3: el entero inventa un orden que no existe	one-hot no inventa nada — y por eso tampoco sabe nada
«una columna densa» de la tabla de más arriba	la tabla de búsqueda entrenable
594 niveles, 0.75 filas por nivel	50.000 columnas con 10.000 registros
Lasso anulando `naviera`	elegir columnas *contra* aprender la geometría

> ***Lleguen con una cosa clara y basta:*** *el costo de una columna de texto no lo fija el texto, lo fija* ***cuántos valores distintos tiene****.*

Para revisar antes del martes

De este cuaderno, tres bloques:

Bloque	Por qué
Q3	el error de codificación más común, y el que mejor explica qué representa un vector
Q5	la cuenta de columnas: 9 → 42 → 37, y de dónde sale cada una
Q6	por qué muchas columnas con pocas filas es un problema, y no una molestia

Las respuestas escritas de Q3, Q4 y Q6 se discuten en clase. Quien quiera retroalimentación individual puede subir su cuaderno; el Laboratorio 2 no tiene entrega obligatoria.

> *El Laboratorio 1 sí se entrega, y tiene sus propias láminas comentadas:* `Semana01_Lab1.pdf`.

Nos vemos el martes

Laboratorio 2 — Unidad 2, Sesión 1

Dr. Carlos Hernández Ulloa
Reiner Romero Garrido

Universidad San Sebastián